

---

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS  
ESCOLA POLITÉCNICA  
ESTRUTURAS DE DADOS 1  
CMP 2106  
PROF. MSC. ANIBAL SANTOS JUKEMURA

# **ESTRUTURAS AUTORREFERENCIADAS (ENCADEADAS)**

## Agenda

- CONCEITOS: Alocação Dinâmica
  - Motivação
- Pilha implementada por estruturas autorreferenciadas
- PILHAS LIGADAS (ENCADEADAS) – IMPLEMENTAÇÃO

## CONCEITOS: Alocação Dinâmica

### Motivação:

- Para representarmos um grupo de dados, já vimos que podemos usar um **vetor**.
- O vetor é a forma mais **primitiva** de representar diversos elementos agrupados.
- O fato de o vetor ocupar um **espaço contíguo** na memória nos permite acessar qualquer um de seus elementos a partir do ponteiro para o primeiro elemento.



## CONCEITOS: Alocação Dinâmica

### Motivação:

- Dizemos que o vetor é uma estrutura que possibilita acesso randômico aos elementos, pois podemos acessar qualquer elemento aleatoriamente (**em tempo constante**).
- o vetor não é uma estrutura de dados muito flexível, pois precisamos dimensioná-lo com um **número máximo** de elementos.



## CONCEITOS: Alocação Dinâmica

### Motivação:

- Não existe uma maneira simples e barata (computacionalmente) para alterarmos a **dimensão** do vetor em tempo de execução.
- Por outro lado, se o número de elementos que precisarmos armazenar no vetor for muito inferior à sua dimensão, estaremos **subutilizando** o espaço de memória reservado.



## CONCEITOS: Alocação Dinâmica

### Motivação:

|         | preco |     |      |    |      |      |
|---------|-------|-----|------|----|------|------|
| células | 9.1   | 4.5 | 75.3 | 84 | 19.2 | 1.88 |
| índices | 0     | 1   | 2    | 3  | 4    | 5    |

↓  
Máximo de Elementos



## CONCEITOS: Alocação Dinâmica

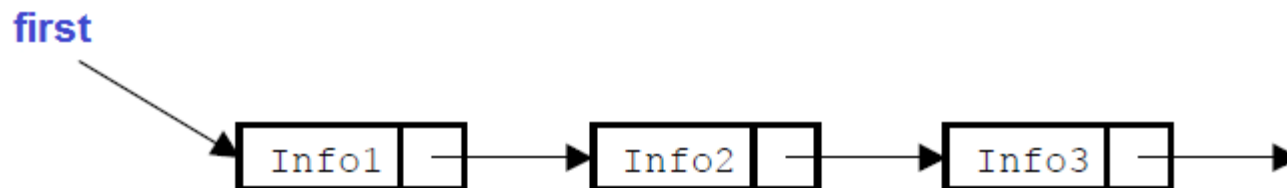
### Motivação:

- A solução para esses problemas é utilizar **estruturas de dados que cresçam** à medida que precisarmos armazenar novos elementos (e **diminuam** à medida que precisarmos retirar elementos armazenados anteriormente). Tais estruturas são chamadas **dinâmicas (autorreferenciadas)** e armazenam cada um dos seus elementos usando alocação dinâmica.



## CONCEITOS: Alocação Dinâmica

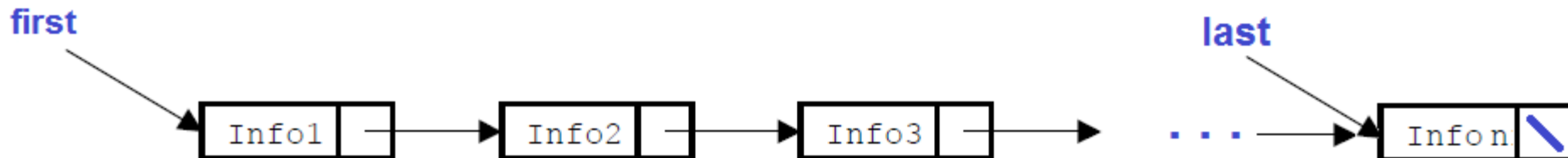
- Para que seja possível percorrer todos os elementos da desse tipo de estrutura, devemos explicitamente guardar o encadeamento dos elementos, o que é feito armazenando-se, junto com a informação de cada elemento, **um ponteiro para o próximo elemento da estrutura**.





## CONCEITOS: Alocação Dinâmica

- A estrutura consiste numa sequência encadeada de elementos, em geral chamados de **nós (nodos)**. A estrutura é representada por um ponteiro para o primeiro elemento (ou nó). Do primeiro elemento, podemos alcançar o segundo seguindo o encadeamento, e assim por diante. O último elemento da lista aponta para **NULL**, sinalizando que não existe um próximo elemento.



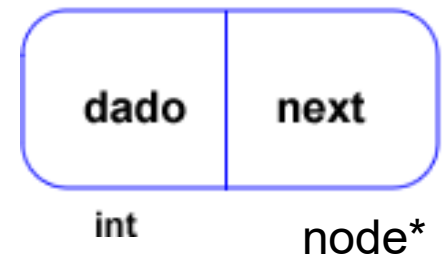
## Pilha implementada por estruturas autorreferenciadas

Padronização:

Para exemplificar a implementação de estruturas encadeadas em C, vamos considerar um exemplo simples em que queremos armazenar valores inteiros numa lista encadeada.

```
typedef struct node {  
    int dado;  
    struct node* next;  
} Node;
```

NÓ ou NODO  
de inteiro



## PILHAS LIGADAS (ENCADEADAS) – IMPLEMENTAÇÃO

### Pilha (Exemplo de pilha de inteiros)

#### Node.h

```
typedef struct node {  
    int dado;  
    struct node* next;  
} Node;
```

```
Node* createNode(int item, Node* ptr);
```

#### Pilha.h

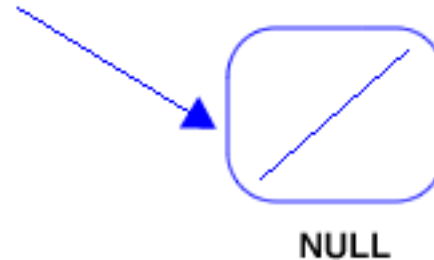
```
void initPilha(Pilha* p);    // Construtor  
void destroyPilha(Pilha* p); // Destrutor  
bool isEmpty(Pilha* p);  
int top(Pilha* p);  
void pop(Pilha* p);  
void makeEmpty(Pilha* p);  
void push(Pilha* p, int elem);  
int topAndPop(Pilha* p);  
int size(Pilha* p);  
void imprimir(Pilha* p);
```

## PILHAS LIGADAS (ENCADEADAS) – IMPLEMENTAÇÃO

### 1. Método de inicialização (**construtor da classe PILHA**)

```
void initPilha(Pilha* p);    // Construtor
```

topo



## PILHAS LIGADAS (ENCADEADAS) – IMPLEMENTAÇÃO

### 2. Método que verifica se a pilha está vazia (**isEmpty**)

```
bool isEmpty(Pilha* p);
```

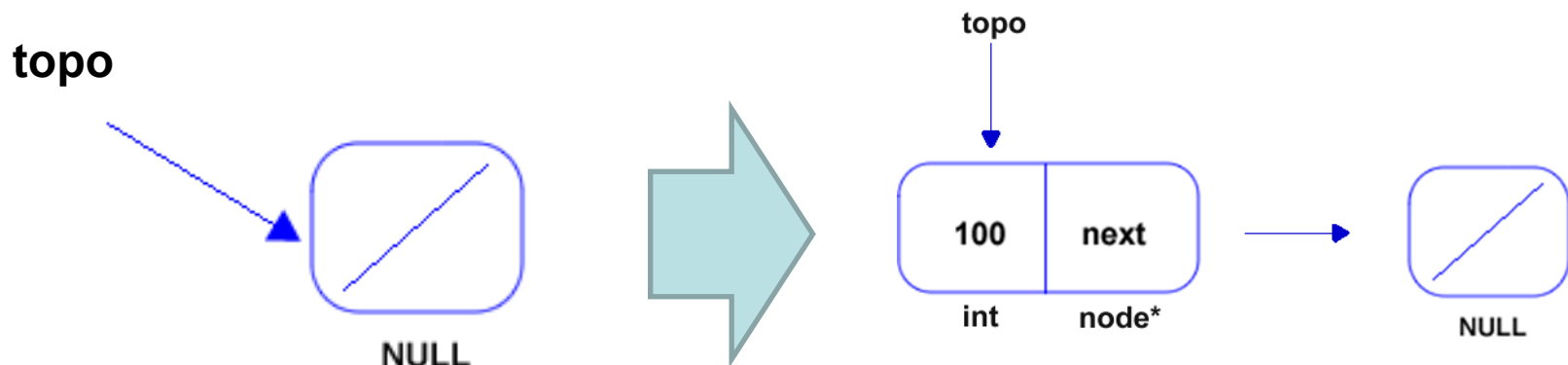
Se o ponteiro topo apontar para NULL, a pilha está vazia.

## PILHAS LIGADAS (ENCADEADAS) – IMPLEMENTAÇÃO

### 3. Método de inserção (**push**)

```
void push(Pilha* p, int elem);
```

Exemplo: PILHA.push(**100**);

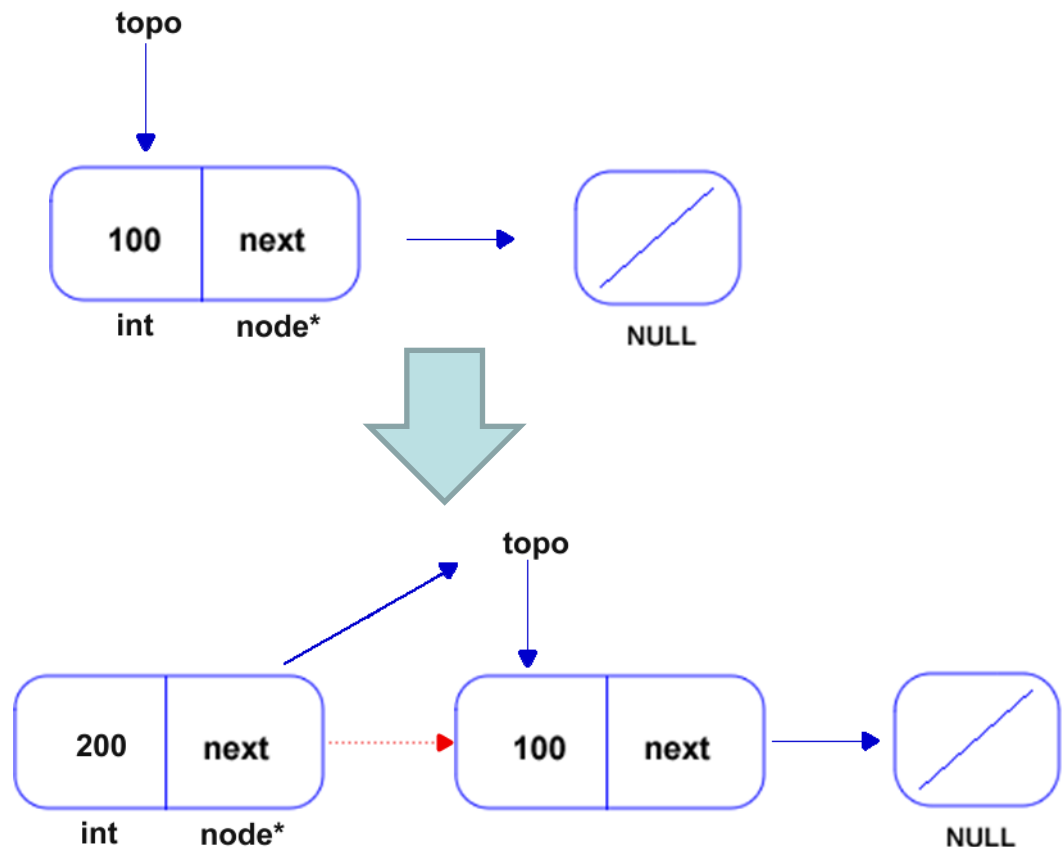


## PILHAS LIGADAS (ENCADEADAS) – IMPLEMENTAÇÃO

### 3. Método de inserção (**push**)

Segunda inserção : Exemplo2: PILHA.push(**200**);

```
void push(Pilha* p, int elem);
```

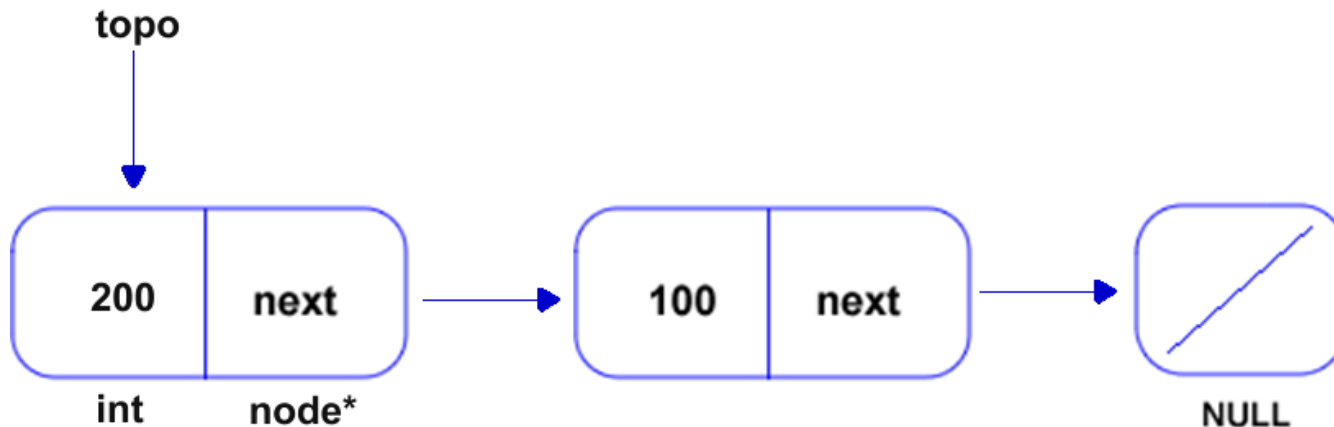


## PILHAS LIGADAS (ENCADEADAS) – IMPLEMENTAÇÃO

### 3. Método de inserção (**push**)

Segunda inserção : Exemplo2: PILHA.push(**push**);

```
void push(Pilha* p, int elem);
```





## PILHAS LIGADAS (ENCADEADAS) – IMPLEMENTAÇÃO

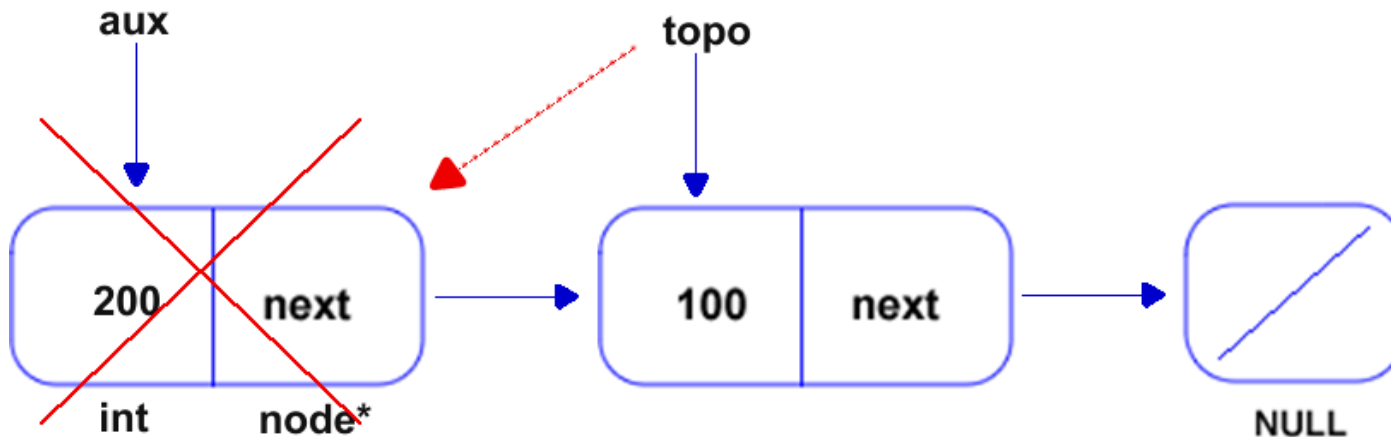
### 4. Método que retorna o tamanho da pilha (**size**)

```
int size(Pilha* p);
```

## PILHAS LIGADAS (ENCADEADAS) – IMPLEMENTAÇÃO

### 5. Método de remoção (**pop**)

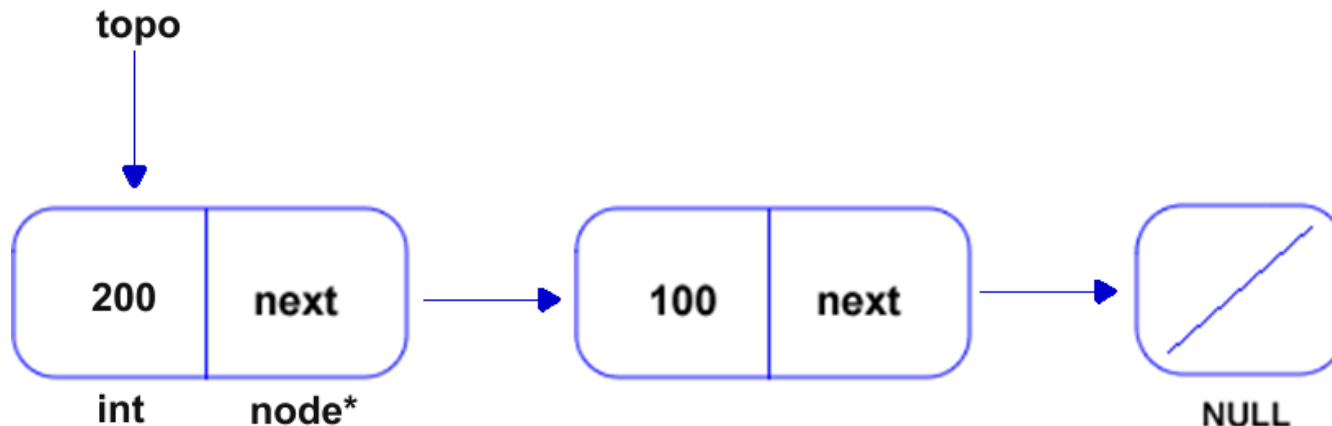
```
void pop(Pilha* p);
```



## PILHAS LIGADAS (ENCADEADAS) – IMPLEMENTAÇÃO

### 6. Método que retorna o topo da pilha(**top**)

```
int top(Pilha* p);
```



## PILHAS LIGADAS (ENCADEADAS) – IMPLEMENTAÇÃO

### 7. Imprime a pilha (**imprimir**)

Passos método: pilha:: **imprimir** ()

- Testar se a lista está vazia antes.
1. Crie um ponteiro auxiliar e faça-o receber topo;
  2. Faça um laço para varrer toda a pilha até que auxiliar==NULL(fim da pilha). Enquanto isso vá imprimindo auxiliar->dado na tela.
  3. Faça auxiliar apontar para NULL;
  4. Remova o ponteiro auxiliar.

## LISTAS SIMPLEMENTE LIGADAS (ENCADEADAS)

### 2ª IMPLEMENTAÇÃO DE ESTRUTURAS DE DADOS

PILHAS SIMPLEMENTE ENCADEADAS  
(SINGLY LINKED STACKS)  
IMPLEMENTAR OS DEMAIS MÉTODOS



## Referências bibliográficas:

- [1]. Pícollo, L. Homero. *Estruturas de Dados*. MSD Ed.. Brasília, 2000.
- [2]. Knuth, D.E. *The Art of Computer Programming - Vols I e III*. 2nd Edition. Addison Wesley, 1973.